Exercise 9 VLSI Testing

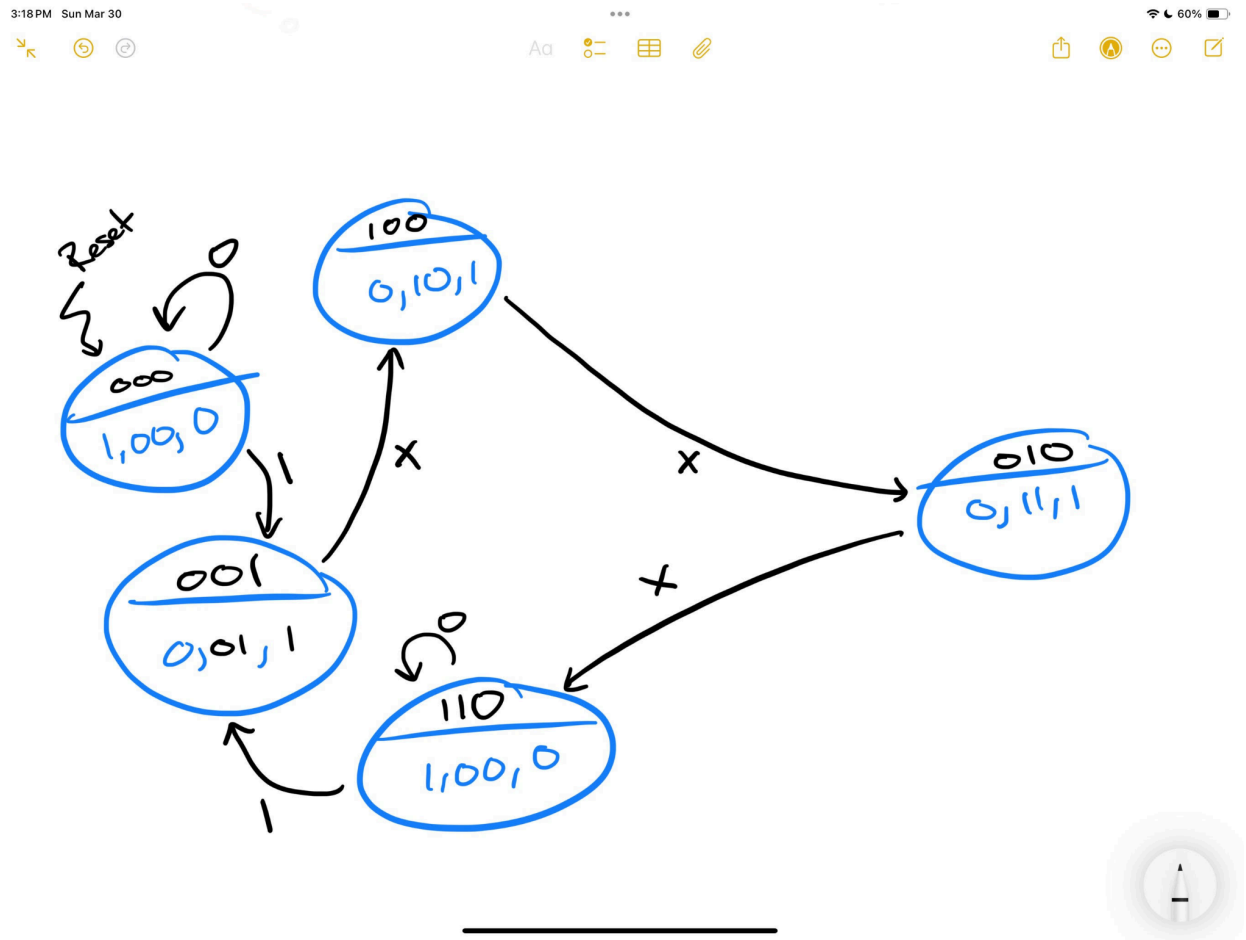
Task 1 - Scan Chain Interface

Checked off by TA

Task 2 - Verifying the Adder

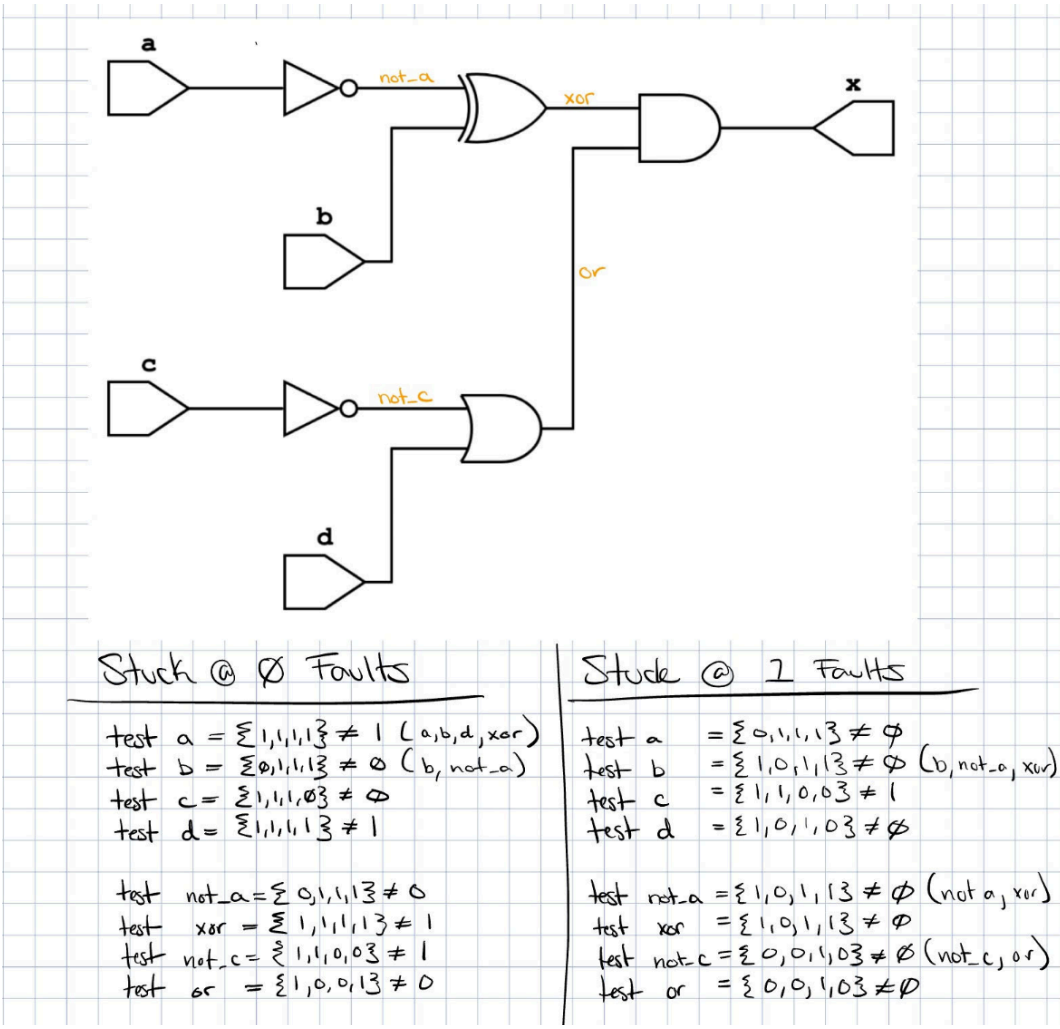
In repo

Task 3 - Reverse Engineer an FSM



For me, as I thought about how to automate the entire process of reverse engineering the FSM. I realized a lot of these algorithms are different flavors of graph processing and backtracking. I thought it would be interesting to create a general FSM reverse engineering algorithm.

Task 4 - Fault Models



The minimum test vector is the test_fault_detection(...) function
Output of the test vectors is located in the same folder as the SV files. Each corresponding output is label faultX.txt. I created the vectors based on overlapping patterns and ability to distinguish which branch (top or bottom) the fault came from. I am a little unconfident about my vectors. The same two caught all the faults for the circuit so…

Task 5 - Reflection

1. The first one that comes to mind is short circuits, so two connecting wires. Assuming we are only testing for two wire short circuits, you could test for bit patterns on a range of input vectors. It would be fairly similar to single fault detection. The other could be open circuit faults. In this case, open circuit faults are unpredictable compared to stuck faults because the wire/pin is floating. As a result, I could imagine a test where you repeatedly probe the potential paths and over a significantly large number of tests you should see

non-deterministic behavior on that path. I would think this test may happen after searching for stuck-at-X faults.

2. The tradeoffs that immediately arise are area, clock speed, power, and potentially security. By adding more combination logic to all paths, the clock speed is decreased, area increases and power increases. Additionally, because you are giving access to the internal state of the chip, it may be easier to reverse engineer. On the flip side, for extremely safety/reliability concerned applications, scan chains can create a more thorough level of verification of chips and reduced debug times.
3. For scenarios as mentioned, this seems like a good candidate for the built-in self testing method since everything can be internal without requiring any external probing and or exposing vulnerabilities.
4. I could imagine that during routing and clock tree synthesis the extra component will significantly affect the clock skew and place and route locations. For companies, I'd imagine they want to bias their designs to be better for the functional mode of the circuit and would be okay to have a more inefficient scan chain test mode. To still create an ASIC that is tailored for its normal operation (trying to get the last bit of performance), you could pass the notion of scan chain construct to their algorithms. By making the algorithms aware, they could choose decisions that are less optimal for scan chain but more optimal for the normal operation mode of the ASIC

Task 6 - Status Update

This week, my goal was to have a working RTL implementation of one of the monitor modules. Both modules are quite similar, with the only difference being the control signals they output. To move forward, I implemented the FSMs for the vertical and horizontal syncs and wrapped them around the Source Input Signal Monitor. Upon reflection, I think it might be more resource-efficient to implement the FSMs primarily as combinational logic, since they aren't overly complex. However, I'll revisit that decision later. Finally, I began integrating the Output Signal Monitor and working on the communication between the two modules. I haven't written detailed test benches for them yet